

Razvoj web aplikacija - Predavanje 11

Testiranje full-stack aplikacije

mag.ing. Josip Torić (SIT UNIZD)

21.12.2026.



Plan predavanja

- 1 Zašto testiramo i piramida testiranja
- 2 pytest: fixtures, conftest i izolacija testova
- 3 Testiranje vremenskih pravila (freezegun)
- 4 Coverage: koliko je dovoljno
- 5 Vitest: unit testovi za storeove i utile
- 6 Testiranje komponenti (@vue/test-utils)
- 7 E2E testovi s Playwrightom
- 8 Testovi u CI-ju i commitovi

Na kraju predavanja: backend i frontend imaju testove koji čuvaju poslovna pravila, a vi znate koji tip testa gdje pripada.

Ishodi učenja

Nakon ovog predavanja student može:

- objasniti piramidu testiranja i odabrati pravi tip testa za problem
- napisati pytest fixture i zamijeniti dependency s `dependency_overrides`
- izolirati testove transakcijom s rollbackom
- testirati vremenski ovisna pravila s freezegunom
- napisati Vitest unit test za Pinia store i utility funkciju
- napisati component test s `@vue/test-utils`
- napisati Playwright E2E test za login i pravilo faze

Kontekst: gdje smo u projektu?

- Backend (P1-P6): API s pravilima, testove smo pisali usput uz svaku temu
- Frontend (P7-P10): SPA s auth flowom, formama i admin ekranima - bez ijednog testa
- Danas: sustavno testiranje **cijelog stacka** prije deploya

Sljedeći korak (Predavanje 12): deploy. Aplikaciju koja ide u produkciju bez testova ne možete mijenjati bez straha - svaka izmjena može nešto potihno razbiti.

Test nije dokaz da kod radi.

Test je dokaz da **pravilo vrijedi** - i da vrijedi i sutra, nakon što nešto promijenite.

Piramida testiranja

Razina	Broj	Brzina	Alat	Što testira
E2E	malo	minute	Playwright	Cijeli flow u browseru
Integration	srednje	sekunde	pytest + httpx	Endpoint kroz sve slojeve
Unit	puno	milisekunde	pytest, Vitest	Izolirana funkcija, store

Što je test niže u piramidi, to je brži i precizniji - kad padne, odmah znate gdje je problem. E2E test hvata ono što unit ne može: integraciju svih dijelova. Zato ih imamo malo, ali na ključnim flowovima.

Anti-pattern je obrnuta piramida: sve kroz spori E2E, ništa kroz unit. Takvi testovi traju predugo i padaju iz nejasnih razloga.

Što smo dosad testirali (rekapitulacija)

Predavanje	Testovi	Što čuvaju
P3	Login, refresh, istekli token	Autentikacija radi i ne otkriva korisnike
P4	Role i cross-club pristup	Klub ne vidi tuđe resurse
P5	CRUD, validacija, duplikati	Constrainti i 409/422 odgovori
P6	Faze s freezegunom	Rokovi zabranjuju akcije

Te testove ste pisali uz gradivo. Danas ih slažemo u strukturu koja se lako održava: zajedničke fixture, izolacija, coverage - i dodajemo frontend koji dosad nije imao ništa.

Anatomija dobrog testa: Arrange - Act - Assert

Svaki test ima tri dijela:

Dio	Što radi
Arrange	Pripremi stanje: korisnik u bazi, token, natjecanje s rokom
Act	Izvrši jednu akciju: pošalji request, pozovi funkciju
Assert	Provjeri ishod: status kod, sadržaj odgovora, stanje u bazi

Ime testa opisuje pravilo, ne implementaciju:

`test_klub_ne_moze_prijaviti_nakon_prelim_roka` govori što je pravilo.
Kad taj test padne, iz imena znate koje se pravilo prekršilo.

Jedan test = jedno pravilo. Test koji provjerava pet stvari padne, a vi ne znate koja od pet je kriva.

pytest fixtures: gradivni blokovi testova

Fixture je funkcija koja priprema ono što test treba - pytest je poziva automatski po imenu parametra:

```
import pytest

@pytest.fixture
def competition_open():
    return Competition(
        name="Otvoreno prvenstvo",
        prelim_deadline=datetime(2027, 3, 1, tzinfo=timezone.utc),
        final_deadline=datetime(2027, 3, 15, tzinfo=timezone.utc),
    )

def test_faza_prije_prelim_roka(competition_open):
    with freeze_time("2027-02-01"):
        assert get_competition_phase(competition_open) == Phase.OPEN
```

Fixture se definira jednom, koristi u koliko god testova treba. Kad se model promijeni, mijenjate jednu fixture umjesto dvadeset testova.

conftest.py: dijeljene fixture za cijeli projekt

```
# tests/conftest.py
import pytest
from httpx import AsyncClient, ASGITransport
from app.main import app
from app.core.deps import get_db

@pytest.fixture
async def client(db_session):
    # zamijeni pravu bazu test sessionom
    app.dependency_overrides[get_db] = lambda: db_session
    transport = ASGITransport(app=app)
    async with AsyncClient(
        transport=transport, base_url="http://test"
    ) as c:
        yield c
    app.dependency_overrides.clear()
```

conftest.py je poseban file - fixture iz njega su dostupne svim testovima bez importa. dependency_overrides je FastAPI mehanizam: sve što je Depends(get_db) u testu dobiva našu test sesiju umjesto prave baze.

Izolacija testova: transakcija + rollback

```
@pytest.fixture
async def db_session():
    async with engine.connect() as conn:
        trans = await conn.begin()
        session = AsyncSession(bind=conn)
        yield session # test radi u transakciji
        await session.close()
        await trans.rollback() # sve promjene nestaju
```

Svaki test dobiva svježu transakciju koja se na kraju rollbacka. Posljedice:

- testovi ne vide podatke jedni od drugih
- redoslijed izvršavanja ne mijenja rezultat
- nema „očisti bazu” koda na kraju svakog testa

Test koji ovisi o podacima prethodnog testa je tempirana bomba - radi dok netko ne pokrene samo jedan test.

Fixture za tokene: testiranje kao admin i kao klub

```
@pytest.fixture
async def admin_token(client):
    resp = await client.post("/auth/login", json={
        "username": "admin", "password": "admin123"})
    return resp.json()["access_token"]

@pytest.fixture
async def club_token(client):
    resp = await client.post("/auth/login", json={
        "username": "behemot", "password": "klub123"})
    return resp.json()["access_token"]

async def test_klub_ne_moze_kreirati_klub(client, club_token):
    resp = await client.post("/clubs",
        json={"name": "Novi klub"},
        headers={"Authorization": f"Bearer {club_token}"})
    assert resp.status_code == 403
```

Za svaki endpoint pišemo barem jedan negativni test: pogrešna rola, tuđi resurs ili istekli rok.

freezegun: vremenska pravila (rekapitulacija P6)

```
from freezegun import freeze_time

@freeze_time("2027-03-05") # između preliminarne i finalne runde
async def test_nova_prijava_nakon_prelim_runda(client, club_token):
    resp = await client.post(
        "/competitions/1/registrations",
        json={"lifter_id": 1, "category": "93"},
        headers={"Authorization": f"Bearer {club_token}"})
    assert resp.status_code == 400
    assert resp.json()["code"] == "deadline_passed"

@freeze_time("2027-03-05")
async def test_odjava_između_rokova_prolazi(client, club_token):
    resp = await client.post(
        "/competitions/1/registrations/1/withdraw",
        headers={"Authorization": f"Bearer {club_token}"})
    assert resp.status_code == 200
```

`freeze_time` zamrzava `datetime.now()` na zadani trenutak. Bez toga bi testovi faza prolazili ili padali ovisno o stvarnom datumu pokretanja.

Coverage: koliko je koda pokriveno

```
uv pip install pytest-cov
pytest --cov=app --cov-report=term-missing

# Name Stmts Miss Cover Missing
# app/services/registration.py 45 3 93% 88-90
# app/routers/clubs.py 30 0 100%
```

Missing stupac pokazuje linije koje nijedan test ne izvršava - često su to error grane i edge caseovi, upravo ono što se najviše isplati testirati.

Coverage mjeri **izvršenost**, ne ispravnost: linija može biti pokrivena testom koji ništa ne provjerava. 100% coverage nije cilj - cilj je da su **poslovna pravila i error grane** pokriveni smislenim assertima. Za ovaj projekt: services i pravila blizu 100%, boilerplate može niže.

Backend: što mora biti pokriveno prije obrane

Područje	Minimum testova
Autentikacija	uspješan login, kriva lozinka, istekli token, re-fresh
Autorizacija	svaka rola na svakom endpointu - tko smije, tko dobiva 403
Ownership	klub A ne vidi i ne mijenja resurse kluba B
Poslovna pravila	svaka faza $\times$ svaka operacija iz matrice
Constrainti	dupla prijava vraća 409, nevažeći input 422

Na obrani pitam: „Što točno testira ovaj test i što bi se moglo pokvariti da ga nema?” Test bez odgovora na to pitanje je balast.

Zašto testirati frontend

Backend ima pytest - frontend mora imati ekvivalent. Bez testova:

- Bug u `parseValidationErrors` otkrije se na obrani, ne lokalno
- Refactoring Pinia storea razbije nešto u formi - ne znate dok ne kliknete
- CI pipeline može pustiti broken kod na produkciju

Tip	Što testira	Alat
Unit	Izolirana funkcija ili store	Vitest
Komponenta	Renderiranje s propsima	Vitest + test-utils
E2E	Cijeli flow u browseru	Playwright

Vitest: setup

```
# Instalacija
npm install -D vitest @vue/test-utils jsdom

# package.json - dodaj skripte
"test": "vitest",
"test:ui": "vitest --ui"

// vite.config.ts - dodaj test konfiguraciju
export default defineConfig({
  plugins: [vue()],
  test: {
    environment: 'jsdom', // simulira browser DOM
    globals: true, // describe/it/expect bez importa
  },
})
```

Vitest koristi istu Vite konfiguraciju - nema zasebnog config filea. API je Jest-kompatibilan: describe, it, expect, vi.mock.

Što testirati unit testovima

Isplati se testirati	Primjer	Zašto
Utility funkcije	<code>parseValidationErrors</code>	Čista logika, brzi testovi
Pinia store actions	<code>fetchLifters</code>	Logika s mock API-jem
Computed getteri	Filtrirani natjecatelji	Derivirana logika

Ne testirati:

- Vue framework sam (reaktivnost, direktive) - Vueov posao
- Third-party biblioteke (Axios, Pinia) - već imaju testove
- Čisti HTML prikaz bez logike - mala vrijednost, veliki maintenance

Unit test: parseValidationErrors

```
// src/utils/errors.test.ts
import { parseValidationErrors } from './errors'

describe('parseValidationErrors', () => {
  it('mapira FastAPI 422 detail na greske po polju', () => {
    const data = {
      detail: [
        { loc: ['body', 'first_name'],
          msg: 'field required', type: 'missing' },
        { loc: ['body', 'birth_date'],
          msg: 'invalid date', type: 'value_error' },
      ]
    }
    expect(parseValidationErrors(data)).toEqual({
      first_name: 'field required',
      birth_date: 'invalid date',
    })
  })

  it('vraca prazan objekt za ne-422 odgovor', () => {
    expect(parseValidationErrors({})).toEqual({})
  })
})
```

Unit test: Pinia store

```
// src/stores/lifters.test.ts
import { setActivePinia, createPinia } from 'pinia'
import { useLifterStore } from './lifters'
import api from '@api/client'

vi.mock('@api/client') // zamijeni Axios s mockom

beforeEach(() => setActivePinia(createPinia()))

it('fetchLifters popunjava state uspjesno', async () => {
  const mockLifter = { id: 1, first_name: 'Ivan' }
  vi.mocked(api.get).mockResolvedValue(
    { data: [mockLifter] }
  )

  const store = useLifterStore()
  await store.fetchLifters(1)

  expect(store.lifters).toHaveLength(1)
  expect(store.lifters[0].first_name).toBe('Ivan')
  expect(store.loading).toBe(false)
  expect(store.error).toBeNull()
})
```

Component test: FormField

```
// src/components/FormField.test.ts
import { mount } from '@vue/test-utils'
import FormField from './FormField.vue'

it('prikazuje label i error poruku', () => {
  const wrapper = mount(FormField, {
    props: { label: 'Ime', error: 'Obavezno polje',
      modelName: '' }
  })
  expect(wrapper.text()).toContain('Ime')
  expect(wrapper.find('.error').text()).toBe('Obavezno polje')
})

it('emitira update:modelValue pri unosu', async () => {
  const wrapper = mount(FormField, {
    props: { label: 'Ime', modelName: '' }
  })
  await wrapper.find('input').setValue('Marko')
  expect(wrapper.emitted('update:modelValue')![0])
    .toEqual(['Marko'])
})
```

mount renderira komponentu u jsdom-u. Testiramo **ugovor** komponente: propsi ulaze, eventi izlaze - ne unutarnju implementaciju.

E2E testovi: Playwright

Playwright kontrolira pravi browser - testira što korisnik zapravo vidi i radi:

- Pokreće Chromium, Firefox ili WebKit
- Navigira, klikće, ispunjava forme - kao pravi korisnik
- Provjera: URL, tekst na stranici, vidljivost elemenata
- Automatski čeka da elementi budu dostupni (nema sleep/timeout)

```
npm install -D @playwright/test  
npx playwright install # preuzme browsere
```

U komponentama dodajte data-testid attribute za stabilne selectore:
data-testid="username" ne puca kad promijenite CSS klasu ili tekst labela.

E2E test: login flow

```
// tests/e2e/login.spec.ts
import { test, expect } from '@playwright/test'

test('prijava preusmjerava na dashboard', async ({ page }) => {
  await page.goto('/login')

  await page.getByTestId('username').fill('behemot')
  await page.getByTestId('password').fill('klub123')
  await page.getByTestId('submit').click()

  await expect(page).toHaveURL('/dashboard')
  await expect(
    page.getByText('Dobro dosli')
  ).toBeVisible()
})

test('pogresna lozinka prikazuje gresku', async ({ page }) => {
  await page.goto('/login')
  await page.getByTestId('username').fill('behemot')
  await page.getByTestId('password').fill('krivo')
  await page.getByTestId('submit').click()
  await expect(page.getByText('Pogresni podaci')).toBeVisible()
})
```

E2E test: pravilo faze u UI-ju

Seed sadrži natjecanje kojem je final rok prošao. E2E test provjerava da UI poštuje fazu:

```
// tests/e2e/phases.spec.ts
import { test, expect } from '@playwright/test'
import { loginAsClub } from './helpers'

test('nakon final roka gumb odjave je disabled',
  async ({ page }) => {
    await loginAsClub(page)
    await page.goto('/competitions/2/registrations')

    await expect(
      page.getByTestId('withdraw-btn')
    ).toBeDisabled()
  })
```

Ovaj test spaja sve: backend vraća faze, frontend computed ih pretvara u `:disabled`, a test dokazuje da korisnik nakon roka stvarno ne može kliknuti. Unit test to ne može pokriti - zato E2E postoji.

Playwright config: automatski dev server

```
// playwright.config.ts
import { defineConfig } from '@playwright/test'

export default defineConfig({
  testDir: 'tests/e2e',
  use: { baseURL: 'http://localhost:5173' },
  webServer: {
    command: 'npm run dev',
    port: 5173,
    reuseExistingServer: true,
  },
})
```

webServer pokreće Vite dev server prije testova i ugasi ga nakon njih. Backend i baza moraju biti pokrenuti zasebno - E2E testira protiv pravog API-ja sa seed podacima.

Zato E2E testove pokrećemo protiv **poznatog stanja**: `python -m app.seed` prije test runa vraća bazu u početno stanje.

Testovi u CI pipelineu

Test	Kad se pokreće	Zašto
pytest	Svaki push	Poslovna pravila backenda
Vitest	Svaki push	Brz (sekunde), hvatamo regresije rano
Playwright	Pull request	Sporiji (minute), treba running server

- `pytest` i `npm run test -- --run` idu u svaki CI run
- Ako test padne - commit dobije crveni X, merge je blokiran
- Cijeli pipeline (lint + test + build) slažemo na Predavanju 12

Za studentski projekt obavezni su `pytest` i `Vitest`. `Playwright` je bonus koji pokazuje zrelost projekta na obrani.

Commitovi za Predavanje 11

- `test(api)`: add `conftest` with `async client` and `db fixtures`
- `test(api)`: add `coverage config` and `fill gaps in rule tests`
- `test(web)`: add `vitest setup` with `util` and `store unit tests`
- `test(web)`: add `component tests` for `form field`
- `test(e2e)`: add `playwright login` and `phase rule tests`

Na satu: pokrećemo testove, rušimo jedno poslovno pravilo namjerno i gledamo koji test pada i zašto.

Što ide sljedeće (Predavanje 12)

- Production konfiguracija i environment varijable
- CORS i deploy backenda na Railway
- Deploy frontenda i SPA fallback
- CI/CD pipeline koji pokreće današnje testove

Pitanja?